

Python & AI Interview Tasks

Candidate Instructions

Complete the assigned task using Python, JS or any preferred programming languages. Unless a task explicitly requires a particular library, you may choose appropriate libraries and frameworks. Prioritize correctness, clean structure, reproducibility, input validation, and clear documentation. Do not hard-code outputs for the supplied examples.

General expectations

- Use Python 3.x and provide a requirements.txt or equivalent dependency specification.
- Keep data loading, preprocessing, model/AI logic, and output handling reasonably separated.
- Include meaningful error handling for invalid input, missing files, or unsupported cases.
- Provide a README with setup steps and commands to run the solution.
- Include a small set of test cases or example inputs and outputs.
- For ML/AI tasks, explain important design choices and evaluation metrics rather than reporting only a final score.

Important: The assignments intentionally cover different areas of Python, machine learning, generative AI, RAG, agents, data processing, forecasting, and human review workflows. Candidates should focus on the requirements stated in each assignment

Task 1 — House Rent Prediction System

Build a Python machine-learning application that predicts monthly house rent from property and location information. The goal is to demonstrate data preparation, feature engineering, model training, evaluation, and prediction on unseen records.

Input : A CSV dataset containing fields such as area/square-feet, number of bedrooms, bathrooms, floor number, property type, furnishing status, locality/city, parking availability, age of property, and monthly rent. Include a separate set of records without the rent column for final prediction.

Required Work

- Load and inspect the dataset using pandas.
- Handle missing values and categorical features appropriately.
- Perform basic exploratory data analysis and identify important features.
- Split the labeled data into training and validation/test sets.
- Train a suitable regression model and justify the choice.
- Save the trained model and create a prediction function or small CLI/API.

Expected Output :

For each unseen property, return a predicted monthly rent. Also provide evaluation metrics, a short explanation of the selected features/model, and at least one visualization showing prediction quality or feature relationships.

Submission / Deliverables :

Python source code, requirements.txt, sample input/output, evaluation results, and a short README describing how to run the solution.

Task 2 — Handwritten Digit Recognition

Create a Python image-classification program that recognizes handwritten digits from 0 to 9. The candidate should demonstrate image preprocessing, supervised learning, model evaluation, and inference on new images.

Input

A labeled handwritten-digit dataset such as MNIST or an equivalent CSV/image dataset. Provide one or more separate handwritten image files for final inference.

Required Work

- Normalize and preprocess the images into a model-compatible representation.
- Split the dataset into training and test data if the dataset is not already split.
- Train a classifier using either a classical ML approach or a neural-network approach.
- Measure accuracy and generate a confusion matrix.
- Create a prediction function that accepts a single image and returns the predicted digit and confidence/probability when supported.
- Handle an invalid or unreadable image gracefully.

Expected Output

For every test image, output the predicted digit. The program should also report overall test accuracy and provide a confusion matrix or equivalent class-wise evaluation.

Submission / Deliverables

Python code, trained model or reproducible training script, example predictions, evaluation metrics, and setup instructions.

Task 3 — Climate / Temperature Prediction

Develop a time-series prediction pipeline that forecasts a future climate variable such as daily temperature. The solution should use historical observations and avoid data leakage from future records.

Input

A chronological CSV containing date/time, temperature, humidity, rainfall, wind speed, pressure, and optionally location or seasonal information. The final portion of the data should be reserved for forecasting evaluation.

Required Work

- Parse dates correctly and sort the observations chronologically.
- Clean missing or anomalous observations.
- Create useful temporal features such as day, month, season, rolling averages, and lag values.
- Train a forecasting/regression model using only information available before the prediction time.
- Evaluate forecasts on a held-out future period.
- Produce predictions for the next N days, where N is clearly stated by the candidate.
- Explain at least two factors that influence the forecast.

Expected Output

A table containing forecast date, predicted temperature, and—where supported—an uncertainty/confidence measure. Include MAE/RMSE on the held-out period and a line chart comparing actual versus predicted values.

Submission / Deliverables

Python project, forecast output CSV, evaluation chart, metrics, and README

Task 4 — Historical Weather Report Generator

Create a Python program that reads historical weather data and automatically produces a human-readable weather report for a selected location and date range.

Input

A CSV containing date, location, temperature, humidity, rainfall, wind speed, and weather condition. The user should provide a location and date range as input.

Required Work

- Filter the dataset by location and requested date range.
- Calculate daily and period-level statistics such as average temperature, maximum/minimum temperature, total rainfall, and average humidity.
- Identify the hottest, coldest, wettest, and windiest days.
- Detect simple patterns or anomalies using clearly stated rules.
- Generate a concise report from the computed values using Python. A template-based solution is acceptable; an optional LLM-generated narrative may be added.

Expected Output

A structured report containing summary statistics, notable dates, trends, and a short narrative such as: “During the selected period, rainfall was above/below the period average...” The report must be traceable to the source data.

Submission / Deliverables

Python code, sample dataset, generated report, and an explanation of the calculations used.

Task 5 — Document-Based RAG Search Assistant

Build a Retrieval-Augmented Generation (RAG) question-answering system that answers questions from a collection of documents rather than relying only on the language model's general knowledge.

Input: A folder containing 5–20 text, Markdown, or PDF documents on a chosen topic. Candidate questions should include questions whose answers are present in the documents and at least one question whose answer is not supported by the documents.

Required Work

- Load and clean the documents.
- Split documents into meaningful chunks and preserve useful metadata such as filename and page/section where available.
- Create embeddings and store them in a vector index/database.
- Retrieve the most relevant chunks for a user query.
- Generate an answer using only the retrieved context.
- Return source references with the answer.
- When evidence is insufficient, explicitly state that the documents do not contain enough information instead of inventing an answer.

Expected Output

For each user question, return: answer, retrieved source/chunk references, and a simple relevance/confidence indication. The system should demonstrate correct behavior for both answerable and unanswerable questions.

Submission / Deliverables

Python code, sample documents, vector-store setup, example queries/results, and README. The candidate may use an open-source or API-based embedding/LLM implementation.

Task 6 — Multi-Step AI Agent Workflow

Objective

Build a Python-based AI agent that receives a user's request, understands what the user wants, decides which tool or tools are required, executes those tools in the correct order, and generates a final response.

The main objective is to demonstrate that you understand the difference between a normal LLM chatbot and an **AI agent that can use tools and perform multi-step tasks**.

The LLM should not directly invent results when the required information is available through a tool.

Scenario

You are building an AI assistant that can answer questions about weather data and perform calculations.

The assistant will have access to several tools:

1. Weather Search
2. Weather Statistics
3. Calculator
4. Text Summarizer

The user can ask simple questions that require one tool or more complex questions that require multiple tools.

Input

The application should accept a natural-language user request.

Examples:

What is the temperature in Hyderabad?

What is the average temperature in Hyderabad?

What is the difference between the temperature in Hyderabad and Chennai?

Compare the temperature and humidity of Hyderabad and Visakhapatnam.

The agent should determine which tools are required for each request.

Required Tools

1. Weather Search Tool

Create a tool that searches the supplied weather CSV dataset.

Use:

06_agent_weather_tool_data.csv

The tool should accept a city name.

Example:

```
{  
  "city": "Hyderabad"  
}
```

Example result:

```
{  
  "city": "Hyderabad",  
  "date": "2026-07-01",  
  "temperature_c": 31.2,  
  "rainfall_mm": 4.2,  
  "humidity_pct": 72  
}
```


The candidate must use the supplied dataset instead of hard-coding weather values.

2. Weather Statistics Tool

Create a tool that calculates weather statistics from the CSV dataset.

The tool should support at least:

average_temperature
maximum_temperature
minimum_temperature
total_rainfall
average_humidity

Example:

```
{  
  "city": "Hyderabad",  
  "metric": "average_temperature"  
}
```

Example result:

```
{  
  "city": "Hyderabad",  
  "metric": "average_temperature",  
  "value": 31.65  
}
```

3. Calculator Tool

Create a calculator tool that performs basic mathematical operations.

It should support:

+
-
*
/

Example:

```
{  
  "expression": "31.2 - 33.0"  
}
```

Expected result:

```
{  
  "result": -1.8  
}
```

The candidate should validate the expression and should **not blindly use** `eval()` on **user input**.

4. Text Summarizer Tool

Create a tool that uses an LLM to summarize supplied text.

Example input:

```
{  
  "text": "Hyderabad recorded 31.2°C while Chennai recorded 33.0°C..."  
}
```

Example output:

```
{  
  "summary": "Chennai recorded a higher temperature than Hyderabad."  
}
```

Agent Requirements

The candidate must implement an agent/controller that:

1. Receives the user's request.
 2. Understands the user's intent.
 3. Selects the appropriate tool.
 4. Generates valid tool arguments.
 5. Executes the selected tool.
 6. Reads the tool result.
 7. Determines whether another tool is required.
 8. Executes additional tools when necessary.
 9. Maintains a short execution state/history.
 10. Generates the final response using the collected results.
-

Example 1 — Single Tool Execution

User

What is the temperature in Hyderabad?

Expected Workflow

User Request
↓
LLM / Agent
↓
Weather Search
↓
Weather Result
↓
Final Response

Expected Tool Call

```
{  
  "tool": "weather_search",  
  "arguments": {  
    "city": "Hyderabad"
```

```
}  
}
```

Expected Response

The recorded temperature in Hyderabad is 31.2°C.

Example 2 — Multi-Step Workflow

User

What is the difference between the temperature in Hyderabad and Chennai?

The agent should recognize that multiple steps are required.

Expected Workflow

User Request

↓

Weather Search — Hyderabad

↓

Weather Search — Chennai

↓

Calculator

↓

Final LLM Response

Example:

Hyderabad = 31.2°C

Chennai = 33.0°C

The calculator should calculate:

$31.2 - 33.0 = -1.8$

Final response:

Chennai is 1.8°C warmer than Hyderabad.

Example 3 — Multiple Tool Calls

User

Compare the temperature and humidity of Hyderabad and Visakhapatnam.

The agent should retrieve information for both cities.

Expected workflow:

Weather Search — Hyderabad



Weather Search — Visakhapatnam



Compare Results



Final Response

Expected response should contain:

Hyderabad:

Temperature: 31.2°C

Humidity: 72%

Visakhapatnam:

Temperature: 29.4°C

Humidity: 78%

The response should clearly explain the comparison.

Example 4 — Tool Chaining

User

Find the temperature in Hyderabad and Chennai, calculate their difference, and summarize the result.

Expected workflow:

Weather Search — Hyderabad



Weather Search — Chennai



Calculator



Text Summarizer



Final Response

The agent should pass the output of one tool to the next tool.

For example:

Hyderabad = 31.2°C

Chennai = 33.0°C

Difference = 1.8°C

Final response:

Chennai is 1.8°C warmer than Hyderabad.

Execution Trace

The application should provide a concise execution trace.

Example:

USER REQUEST

What is the difference between Hyderabad
and Chennai temperature?

STEP 1

Tool: weather_search
Arguments: {"city": "Hyderabad"}
Result: 31.2°C

STEP 2

Tool: weather_search
Arguments: {"city": "Chennai"}
Result: 33.0°C

STEP 3

Tool: calculator
Arguments: {"expression": "31.2 - 33.0"}
Result: -1.8

FINAL RESPONSE

Chennai is 1.8°C warmer than Hyderabad.

The candidate does not need to expose the LLM's private chain-of-thought. Only the **tool name, arguments, results, and execution order** should be shown.

Test Dataset

Use the following dataset:

06_agent_weather_tool_data.csv

city	date	temperature_c	rainfall_mm	humidity_pct
Hyderabad	2026-07-01	31.2	4.2	72
Hyderabad	2026-07-02	32.1	0.0	65

Vijayawada,2026-07-01,34.5,2.1,61
Visakhapatnam,2026-07-01,29.4,8.7,78
Chennai,2026-07-01,33.0,3.0,70

Required Test Cases

Test Case 1 — Weather Search

Input:

What is the temperature in Hyderabad?

Expected tool:

weather_search

Expected temperature:

31.2°C

Test Case 2 — Different City

Input:

What is the temperature in Chennai?

Expected:

33.0°C

Test Case 3 — Weather Statistics

Input:

What is the average temperature in Hyderabad?

Expected tool:

weather_statistics

Expected result:

31.65°C

Test Case 4 — Multi-Step Calculation

Input:

What is the difference between the temperature in Hyderabad and Chennai?

Expected tool sequence:

weather_search
→ weather_search
→ calculator

Expected calculation:

$31.2 - 33.0 = -1.8$

Expected final response:

Chennai is 1.8°C warmer than Hyderabad.

Test Case 5 — Multiple Data Points

Input:

Compare the temperature and humidity of Hyderabad and Visakhapatnam.

Expected tools:

weather_search(Hyderabad)
weather_search(Visakhapatnam)

The final answer should compare both:

Temperature
Humidity

Test Case 6 — Calculator

Input:

If the temperature in Hyderabad is 31.2°C,
what will it be after increasing by 5°C?

Expected:

36.2°C

The agent should use the calculator tool instead of relying on the LLM to perform the calculation.

Test Case 7 — Tool Chaining

Input:

Find the temperature in Hyderabad and Chennai,
calculate their difference, and summarize the result.

Expected workflow:

weather_search

→ weather_search

→ calculator

→ summarizer/final LLM

Expected information:

Hyderabad: 31.2°C

Chennai: 33.0°C

Difference: 1.8°C

Test Case 8 — Unknown City

Input:

What is the temperature in Mumbai?

Mumbai does not exist in the supplied dataset.

Expected behavior:

I don't have weather data for Mumbai
in the supplied dataset.

The agent must **not invent a temperature**.

Test Case 9 — Unsupported Live Data

Input:

What will the weather be tomorrow in London?

The supplied tools do not provide live weather or London weather data.

Expected behavior:

I cannot provide live London weather because the available weather data does not contain it.

The agent must not hallucinate a weather prediction.

Test Case 10 — Invalid Input

Input:

Compare the weather of

The agent should either:

ask the user for the city names

or return a controlled clarification such as:

Please provide the cities you want me to compare.

The application should not crash.

Error Handling Requirements

The candidate must handle:

- Unknown cities
- Missing tool arguments
- Invalid tool arguments
- Invalid calculator expressions
- Empty user messages
- Tool execution failures
- LLM returning invalid tool names
- LLM returning malformed tool arguments

- Maximum agent steps being exceeded
- Unsupported requests

The agent should return a controlled error or clarification instead of crashing or hallucinating an answer.

Tool Definition

Each tool should have:

Tool Name

Description

Input Parameters

Parameter Types

Required Parameters

Return Format

For example:

```
{
  "name": "weather_search",
  "description": "Search weather information for a city.",
  "parameters": {
    "city": {
      "type": "string",
      "description": "Name of the city"
    }
  }
}
```

The candidate may use:

- Native LLM tool/function calling
- Ollama tool calling
- LangChain
- LlamaIndex
- Custom Python implementation
- LangGraph

Using a framework is optional.

State / History

The agent should maintain a short execution history.

Example:

```
[
  {
    "step": 1,
    "tool": "weather_search",
    "arguments": {
      "city": "Hyderabad"
    },
    "result": {
      "temperature_c": 31.2
    }
  },
  {
    "step": 2,
    "tool": "weather_search",
    "arguments": {
      "city": "Chennai"
    },
    "result": {
      "temperature_c": 33.0
    }
  },
  {
    "step": 3,
    "tool": "calculator",
    "arguments": {
      "expression": "31.2 - 33.0"
    },
    "result": -1.8
  }
]
```

This information should be available when generating the final response.

Acceptance Criteria

The task is considered complete when:

- Python application runs successfully.
 - LLM integration works.
 - Weather Search tool is implemented.
 - Weather Statistics tool is implemented.
 - Calculator tool is implemented.
 - Text Summarizer tool is implemented.
 - Tools have clear definitions and input schemas.
 - Agent can select the appropriate tool.
 - Agent can execute multiple tools.
 - Output from one tool can be passed to another.
 - Agent maintains execution state/history.
 - Tool inputs are validated.
 - Tool failures are handled.
 - Unknown information is not hallucinated.
 - Final response is based on tool results.
 - Execution trace is available.
 - At least 5–8 test prompts are implemented.
 - README is provided.
 - `requirements.txt` is provided.
-

Submission / Deliverables

The candidate should submit:

1. Python source code
2. `requirements.txt`
3. `README.md`
4. `06_agent_weather_tool_data.csv`
5. Tool definitions
6. Agent implementation
7. Test cases

8. Sample execution traces
9. Architecture diagram
10. Example outputs

A graphical UI is **not required**. A CLI application is sufficient.

Task 7 — Prompt-Based Text Classifier

Objective

Build a Python application that uses an LLM to classify customer messages into predefined categories. The application must send the customer message to an LLM and receive a **structured JSON response** containing:

- The predicted category
- A confidence score
- Optionally, a short reason for the classification

The main objective is to evaluate the candidate's understanding of:

- Prompt engineering
 - LLM API integration
 - Structured output
 - JSON parsing
 - Input validation
 - Handling unexpected LLM responses
-

Problem Statement

A company receives customer support messages through its website.

Examples: "My order hasn't arrived yet."

"I want to return the shoes I purchased."

"I forgot my password. How can I reset it?"

The system should automatically identify the type of customer request.

Supported Categories

The candidate must use the following categories:

order_issue
return_request
payment_issue
account_issue
product_information
Complaint
other

The LLM should classify every incoming message into exactly one of these categories.

Input

The program should accept a customer message.

Example: My order #ORD12345 hasn't arrived yet.

Another example: I want to return the product I purchased yesterday.

Expected Output

The application should return valid JSON.

Example:

```
{  
  "category": "order_issue",  
  "confidence": 0.95  
}
```

Another valid response:

```
{  
  "category": "return_request",  
  "confidence": 0.91  
}
```

The confidence should be a number between:

0.0 and 1.0

Prompt Requirements

The candidate should design a prompt that clearly tells the LLM:

1. What the task is.
2. Which categories are allowed.
3. What each category means.

4. What output format is required.
5. That the model must not invent categories.

The candidate may use few-shot examples in the prompt.

Required Test Cases

Test Case 1

Input: My order was supposed to arrive yesterday but I still haven't received it.

Expected category:

order_issue

Test Case 2

Input: I don't like the product and would like to return it.

Expected category:

return_request

Test Case 3

Input: My credit card was charged twice for the same order.

Expected category:

payment_issue

Test Case 4

Input: I forgot my password and cannot login to my account.

Expected category:

account_issue

Test Case 5

Input: Can you tell me whether this laptop has 16GB RAM?

Expected category:

product_information

Test Case 6

Input: The product I received is completely damaged and unusable.

Expected category:

complaint

Test Case 7 — Unknown Case

Input: What time does your warehouse close on Sundays?

Expected category:

other

Edge Cases

The candidate should also test:

""

Very long customer messages.

Messages containing emojis.

Messages containing multiple issues.

Messages containing numbers and order IDs.

Example:

Order ORD99881 was damaged and I was also charged twice.

The candidate should explain how the system handles messages containing multiple possible categories.

Acceptance Criteria

The task is considered complete when:

- LLM integration works.
- All seven categories are supported.
- Output is valid JSON.
- Category is restricted to the allowed values.
- Confidence is between 0 and 1.
- Invalid LLM output is handled.
- Empty input is handled.
- API key is hard-coded or not hard-coded.
- Test cases are implemented.
- README is provided.

Bonus

Bonus points if the candidate implements:

- JSON schema validation
- Automatic retry when malformed JSON is returned
- Few-shot prompting
- Logging
- Batch classification
- Unit tests

Task 8 — Text Summarization Tool

Objective

Build a Python application that accepts a long text/article and uses an LLM to generate a structured summary.

The system should produce:

1. A shot summary
2. Five key points
3. Important keywords

This task evaluates:

- LLM prompting
 - Context handling
 - Output formatting
 - Structured generation
 - Text preprocessing
 - Basic LLM application design
-

Input

The application should accept a text document.

Example:

Artificial intelligence has become one of the most important technologies in modern software development. Large language models are being used for customer support, document processing, code generation, data analysis and many other applications. Companies are increasingly integrating AI into existing software systems...

The text may come from:

- User input
- `.txt` file
- Command-line argument

The candidate may choose the interface.

Expected Output

The output should contain three sections.

Summary

Exactly three concise lines.

Key Points

Exactly five points.

Keywords

A list of important keywords.

Example:

SUMMARY

1. Artificial intelligence is increasingly used in software development.
2. LLMs support applications such as customer service and code generation.
3. Businesses are integrating AI into existing software systems.

KEY POINTS

1. AI is becoming important in modern software development.
2. LLMs are used in customer support.
3. AI can assist with document processing.
4. AI is increasingly used for code generation.
5. Companies are integrating AI into existing applications.

KEYWORDS

AI, artificial intelligence, LLM, software development,
customer support, code generation

Important Requirement

The model should summarize **only the supplied text**.

The candidate should avoid allowing the LLM to introduce unrelated information.

Required Test Cases

Test Case 1 — Normal Article

Provide a 300–500 word article about artificial intelligence.

Expected:

3-line summary

5 key points

keywords

Verify that:

- Summary has exactly 3 lines.
 - Key points contain exactly 5 items.
 - Keywords are present.
 - Information comes from the supplied article.
-

Test Case 2 — Short Text

Input: Python is a programming language used for web development,
automation, data science and artificial intelligence.

The application should still produce the requested structure.

Test Case 3 — Unrelated Content

Input:

The Earth revolves around the Sun.
Water freezes at 0 degrees Celsius.
Python is widely used for programming.

The LLM should not introduce unrelated information such as:

Python was created for machine learning.

unless that information actually exists in the input.

Test Case 4 — Empty Input

Input: "" Expected:

Error: Text input cannot be empty.

The application should not unnecessarily call the LLM.

Test Case 5 — Long Document

Provide a document containing several thousand words.

The candidate should demonstrate how the application handles long input.

Possible approaches:

- Chunking
 - Token counting
 - Map-reduce summarization
 - Truncation with a clear explanation
-

Acceptance Criteria

- Accepts text input.
- Produces exactly 3 summary lines.
- Produces exactly 5 key points.
- Produces keywords.
- Handles empty input.
- Handles long input.
- Output format is consistent.
- Does not hard-code responses.
- README is provided.
- Test cases are included.

Bonus

- Support `.txt` files.
- Support PDF input.
- Add word/token limits.
- Add JSON output mode.
- Add summary length configuration.
- Add unit tests.